DAY 4 — ADVANCED JAVA PROGRAMMING CONCEPTS

Completed Tasks: Concepts, Programs, and Sample Output

1. Fundamental Concepts

Arrays: Fixed-size collection of values of the same type, accessed by zero-based index.

Strings: Text objects in Java. Strings are immutable; useful methods include length(), equals(), contains(), and compareToIgnoreCase().

Collections Framework: Classes and interfaces for groups of objects. List preserves order; Map stores key-value pairs.

ArrayList: Resizable list. **HashMap:** Stores values using unique keys, making ID-based lookup convenient.

Exception handling: try/catch handles errors such as invalid numeric input. finally can be used for cleanup.

File handling: Java can read/write files using classes such as Files, Path, BufferedReader and BufferedWriter.

Java Streams: A way to process collections with operations such as filter(), map() and sorted().

2. Main Task — Student Data Processing Application

This program stores student records in an ArrayList and HashMap, searches by ID or name, sorts by name or marks, handles invalid numeric input, and filters students with a Stream.

```
import java.util.*;

class Student {
    private int id;
    private String name;
    private double marks;

    Student(int id, String name, double marks) {
        this.id = id; this.name = name; this.marks = marks;
    }
    int getId() { return id; }
    String getName() { return name; }
    double getMarks() { return marks; }

    void display() {
        System.out.printf("ID: %d | Name: %s | Marks: %.2f%n",
                          id, name, marks);
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        ArrayList<Student> list = new ArrayList<>();
        HashMap<Integer, Student> map = new HashMap<>();
        int choice = -1;

        while (choice != 0) {
            System.out.println("\n1.Add  2.Display  3.Search ID");
            System.out.println("4.Search name  5.Sort name  6.Sort marks");
            System.out.println("7.Marks >= 60 (Stream)  0.Exit");
            System.out.print("Choice: ");
            try {
                choice = sc.nextInt();
                sc.nextLine();
                switch (choice) {
                    case 1:
                        System.out.print("ID: ");
                        int id = sc.nextInt(); sc.nextLine();
                        if (map.containsKey(id)) {
                            System.out.println("ID already exists."); break;
                        }
                        System.out.print("Name: ");
                        String name = sc.nextLine().trim();
                        System.out.print("Marks (0-100): ");
                        double marks = sc.nextDouble(); sc.nextLine();
                        if (name.isEmpty() || marks < 0 || marks > 100) {
                            System.out.println("Invalid name or marks."); break;
                        }
                        Student st = new Student(id, name, marks);
                        list.add(st); map.put(id, st);
                        System.out.println("Student added.");
                        break;
                    case 2:
                        if (list.isEmpty()) System.out.println("No records.");
                        else for (Student x : list) x.display();
                        break;
                    case 3:
                        System.out.print("ID to find: ");
                        int key = sc.nextInt(); sc.nextLine();
                        Student found = map.get(key);
                        if (found == null) System.out.println("Not found.");
                        else found.display();
                        break;
                    case 4:
                        System.out.print("Name to find: ");
                        String q = sc.nextLine().trim();
                        boolean matched = false;
                        for (Student x : list)
```

```
                        if (x.getName().equalsIgnoreCase(q)) {
                            x.display(); matched = true;
                        }
                    if (!matched) System.out.println("Not found.");
                    break;
                case 5:
                    list.sort(Comparator.comparing(
                        Student::getName, String.CASE_INSENSITIVE_ORDER));
                    for (Student x : list) x.display();
                    break;
                case 6:
                    list.sort(Comparator.comparingDouble(
                        Student::getMarks).reversed());
                    for (Student x : list) x.display();
                    break;
                case 7:
                    list.stream().filter(x -> x.getMarks() >= 60)
                        .forEach(Student::display);
                    break;
                case 0: System.out.println("Goodbye."); break;
                default: System.out.println("Invalid menu choice.");
            }
        } catch (InputMismatchException e) {
            System.out.println("Invalid input. Enter the expected type.");
            sc.nextLine();
        }
    }
    sc.close();
  }
}
```

3. Sample Output

```
1.Add  2.Display  3.Search ID
4.Search name  5.Sort name  6.Sort marks
7.Marks >= 60 (Stream)  0.Exit
Choice: 1
ID: 101
Name: Aarav
Marks (0-100): 86
Student added.
Choice: 3
ID to find: 101
ID: 101 | Name: Aarav | Marks: 86.00
```

4. How the Tasks Are Covered

• Store/process records: ArrayList. • Search by ID: HashMap. • Search by name: case-insensitive String comparison. • Sort by name or marks: Comparator. • Invalid input: InputMismatchException. • Stream: filter() selects marks of 60 or above.

5. File Handling Example (Separate Program)

Save this separately as FileDemo.java. It writes to and reads from a text file.

```
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;

public class FileDemo {
    public static void main(String[] args) {
        try {
            Path path = Path.of("students.txt");
            Files.writeString(path, "101,Aarav,86.0");
            System.out.println(Files.readString(path));
        } catch (IOException e) {
            System.out.println("File error: " + e.getMessage());
        }
    }
}
```

6. Compile and Run

Save the main program as Main.java, then run `javac Main.java` and `java Main`. Run FileDemo separately from FileDemo.java. The file example requires Java 11 or later.

Note: The student application keeps records in memory only; records are lost when it exits. The file example demonstrates file I/O separately and does not save the application records.